



Security Audit Report

Nawa USDT stable vault (Ethereum)

v1.0

July 16, 2026

Table of Contents

Table of Contents

License

Disclaimer

Introduction

Purpose of This Report

Codebase Submitted for the Audit

Methodology

Functionality Overview

How to Read This Report

Code Quality Criteria

Summary of Findings

Detailed Findings

1. Zero AUM with live share supply bypasses AUM limits and corrupts share pricing
2. `updateAum` has no staleness bound, so a stalled oracle leaves deposits and redemptions priced on old NAV
3. The documented upgrade exit window is not enforceable on-chain
4. Zero update cooldown lets repeated oracle reports bypass the intended NAV movement rate
5. Loss-report front-running lets redeemers lock pre-loss NAV
6. Admin can retroactively delay the cancel deadline of already submitted redemption requests
7. Stale absolute AUM reports can misaccount post-snapshot state changes
8. Empty-vault bootstrap deposits can be diluted by cancellation of a pending full redemption
9. `mintingCap` does not remain a hard cap across pending redemptions
10. User entry and exit functions lack slippage and deadline protection
11. Just-in-time deposits before positive NAV reports dilute existing holders
12. Redemption approval can change share pricing while the vault is paused
13. Stale funding queue slots can overstate the redemption backlog and delay surplus deployment
14. Zero AUM reports cannot be recorded during empty-vault or total-loss states
15. Initialization and approval paths emit incomplete events, leaving off-chain trackers with gaps
16. No mechanism to burn or freeze wrongly minted shares, upgrade is the only remedy
17. Extreme rounded-to-zero price can brick ordinary AUM updates
18. `BackstopFunded` reports a consumed amount that can exceed the backstop transfer
19. Non-contiguous storage layout increases upgrade maintenance risk
20. No emergency stop can halt outflows if funds are ever credited in error
21. Initialization does not enforce role separation, so one key can hold every privileged role
22. NAV guardrails are admin-governed operational controls

- 23. Deploy default can assign upgrade timelock control to admin
- 24. Configured asset is not validated against USDT assumptions
- 25. Discretionary redemption overrides can shift value between redeemers and remaining holders
- 26. Admin handover does not rotate operator roles, leaving stale powers such as pause

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES
4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED "AS IS", WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io>

info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Highend Technologies LLC to perform a security audit of the Nawa USDT stable vault (Ethereum).

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	URL	Commit
stable-eth-vault	https://github.com/Nawa-Finance/stable-eth-vault	86d1f88bedf4f8ac365a0c02fcd63e48889109b9

Scope	contracts/ NawaUSDTVault.sol
-------	---------------------------------

Fixes Verified at Commit

8da941713ea945ce7f7845026bc5ba32374cea78

Note that only fixes to the issues described in this report have been reviewed at this commit.

Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation.

Functionality Overview

The repository contains a single upgradeable smart contract, `NawaUSDTVault.sol`, implementing a share-based USDT yield vault on Ethereum. Users deposit USDT and receive `nzUSDT`, a non-rebasing ERC-20 receipt token whose value accrues through an increasing price-per-share rather than balance adjustments. Deposited capital is deployed off-chain to a private-credit strategy; the contract itself acts as a transient escrow and router rather than a persistent custodian.

Redemptions follow an asynchronous lifecycle: shares are burned and priced at request time, a designated `redeemManager` approves or rejects requests, and payouts are funded via deposit netting or a multisig-controlled backstop before users pull USDT through a pull-over-push claims ledger. Access control is segmented across seven distinct roles. The contract is deployed behind a UUPS proxy with upgrades gated to a `TimelockController` enforcing a 72-hour delay, and storage layout compatibility is validated prior to queuing any upgrade.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or losses, with medium to high probability.
Minor	A violation of best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects such as user experience or readability. This category may also include opinionated recommendations that the project team might not agree with.

The **status** of an issue can be one of the following: **Resolved**, **Acknowledged**, or **Partially Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium	Compact single-contract design, but async redemptions plus oracle-driven accounting create nontrivial state and invariant complexity.
Code readability and clarity	Medium-High	The code is well-commented and organized, though the accounting model still requires careful tracing.
Level of documentation	Medium-High	Strong architecture and inline documentation, but some deployment/runbook assumptions need tighter coverage.
Test coverage	Medium-High	Broad functional coverage exists, though some edge branches and invariant-style fuzz coverage are still missing.

Summary of Findings

No	Description	Severity	Status
1	Zero AUM with live share supply bypasses AUM limits and corrupts share pricing	Major	Resolved
2	updateAum has no staleness bound, so a stalled oracle leaves deposits and redemptions priced on old NAV	Minor	Resolved
3	The documented upgrade exit window is not enforceable on-chain	Minor	Resolved
4	Zero update cooldown lets repeated oracle reports bypass the intended NAV movement rate	Minor	Acknowledged
5	Loss-report front-running lets redeemers lock pre-loss NAV	Minor	Acknowledged
6	Admin can retroactively delay the cancel deadline of already submitted redemption requests	Minor	Resolved
7	Stale absolute AUM reports can misaccount post-snapshot state changes	Minor	Resolved
8	Empty-vault bootstrap deposits can be diluted by cancellation of a pending full redemption	Minor	Resolved
9	mintingCap does not remain a hard cap across pending redemptions	Minor	Acknowledged
10	User entry and exit functions lack slippage and deadline protection	Minor	Resolved
11	Just-in-time deposits before positive NAV reports dilute existing holders	Minor	Acknowledged
12	Redemption approval can change share pricing while the vault is paused	Minor	Resolved

No	Description	Severity	Status
13	Stale funding queue slots can overstate the redemption backlog and delay surplus deployment	Minor	Resolved
14	Zero AUM reports cannot be recorded during empty-vault or total-loss states	Minor	Resolved
15	Initialization and approval paths emit incomplete events, leaving off-chain trackers with gaps	Informational	Resolved
16	No mechanism to burn or freeze wrongly minted shares, upgrade is the only remedy	Informational	Resolved
17	Extreme rounded-to-zero price can brick ordinary AUM updates	Informational	Resolved
18	BackstopFunded reports a consumed amount that can exceed the backstop transfer	Informational	Resolved
19	Non-contiguous storage layout increases upgrade maintenance risk	Informational	Resolved
20	No emergency stop can halt outflows if funds are ever credited in error	Informational	Resolved
21	Initialization does not enforce role separation, so one key can hold every privileged role	Informational	Resolved
22	NAV guardrails are admin-governed operational controls	Informational	Acknowledged
23	Deploy default can assign upgrade timelock control to admin	Informational	Resolved
24	Configured asset is not validated against USDT assumptions	Informational	Resolved
25	Discretionary redemption overrides can shift value between redeemers and remaining holders	Informational	Resolved

No	Description	Severity	Status
26	Admin handover does not rotate operator roles, leaving stale powers such as pause	Informational	Acknowledged

Detailed Findings

1. Zero AUM with live share supply bypasses AUM limits and corrupts share pricing

Severity: Major

- contracts/NawaUSDTVault.sol:291-305
- contracts/NawaUSDTVault.sol:361-382
- contracts/NawaUSDTVault.sol:499-528
- contracts/NawaUSDTVault.sol:705-714

The NawaUSDTVault contract can enter a state where `latestAum == 0` while `totalSupply() > 0`. This breaks a core accounting invariant: if shares remain outstanding, the vault should not enter a zero-AUM pricing state unless it is in an explicit recovery mode.

The state can be reached through the redemption override flow. In `contracts/NawaUSDTVault.sol:369-373`, `_applyOverride` subtracts the override delta from `latestAum` and only reverts when `delta > latestAum`:

```
if (newAmount > oldAmount) {
    uint256 delta = newAmount - oldAmount;
    if (delta > latestAum) revert AumUnderflow();
    latestAum -= delta;
    pendingRedemptionAum += delta;
}
```

If `delta == latestAum`, `latestAum` becomes zero while shares can remain outstanding.

The same state can also be reached without a privileged override after a sufficiently deep NAV drawdown has been reported. `requestRedeem` subtracts the rounded asset amount from `latestAum` and only clears dust when the redemption burns the full remaining share supply:

```
uint256 assetAmount = _convertToAssets(shares);
if (assetAmount == 0) revert ZeroAmount();
uint256 pps = _pricePerShare();

_burn(msg.sender, shares);

latestAum -= assetAmount;
pendingRedemptionAum += assetAmount;

// When the last holder exits, clear the rounding dust so the vault
// re-bootstraps at pps 1.0 on the next deposit.
if (totalSupply() == 0) latestAum = 0;
```

Because `_convertToAssets` rounds down,

```
function _convertToAssets(uint256 shares) private view returns (uint256) {
    return Math.mulDiv(shares, latestAum + VIRTUAL_ASSETS, totalSupply() +
VIRTUAL_SHARES);
}
```

a sufficiently large partial redemption can round to the entire remaining latestAum while still leaving some shares outstanding. This requires a deep loss state, approximately $\text{latestAum} \leq (\text{totalSupply}() - 1) / 2$, and a holder able to redeem the required share amount. The redemption step itself is unprivileged.

Once $\text{latestAum} == 0 \ \&\& \ \text{totalSupply}() > 0$, updateAum skips the AUM change-limit. In `contracts/NawaUSDTVault.sol:519-523`, the price movement check only runs when $\text{oldAum} > 0$:

```
if (oldAum > 0) {
    uint256 oldPps = Math.mulDiv(oldAum + VIRTUAL_ASSETS, PPS_SCALE, supply +
VIRTUAL_SHARES);
    uint256 candidatePps = Math.mulDiv(newAum + VIRTUAL_ASSETS, PPS_SCALE,
supply + VIRTUAL_SHARES);
    if (_deviationBps(candidatePps, oldPps) > aumChangeLimitBps) revert
ChangeLimitExceeded();
}
```

Therefore, the `aumUpdater` can set any positive newAum in a single update, even if the resulting price-per-share movement is far above `aumChangeLimitBps`.

The same corrupted state also breaks deposit pricing. In `contracts/NawaUSDTVault.sol:705-707`, `_convertToShares` divides by `latestAum + VIRTUAL_ASSETS`:

```
function _convertToShares(uint256 amount) private view returns (uint256) {
    return Math.mulDiv(amount, totalSupply() + VIRTUAL_SHARES, latestAum +
VIRTUAL_ASSETS);
}
```

When $\text{latestAum} == 0$, the denominator is 1. A depositor can therefore mint an excessive number of shares at an artificially low price, heavily diluting existing holders and desynchronizing on-chain share supply from the economic NAV.

Recommendation

We recommend enforcing the invariant that `latestAum` cannot be zero while `totalSupply()` is nonzero, unless the vault enters an explicit recovery mode.

In `_applyOverride`, reject any override that would reduce `latestAum` to zero while shares remain outstanding. In `requestRedeem`, reject any partial redemption that would reduce `latestAum` to zero while `totalSupply()` remains nonzero after the burn. Full redemptions that burn all

remaining supply can continue to zero `latestAum`.

In `deposit`, `requestRedeem`, and other pricing paths, explicitly revert when `latestAum == 0 && totalSupply() > 0`, unless a deliberately designed recovery mode is active.

In `updateAum`, do not silently skip the change-limit solely because `oldAum == 0` while shares remain outstanding. If legitimate recovery from a zero-AUM live-supply state is required, implement a separate authorized recovery mechanism with bounded price movement, explicit events, and clear operational constraints.

Status: Resolved

2. `updateAum` has no staleness bound, so a stalled oracle leaves deposits and redemptions priced on old NAV

Severity: Minor

- `contracts/NawaUSDVault.sol:499-529`
- `contracts/NawaUSDVault.sol:291-320`
- `contracts/NawaUSDVault.sol:258-287`

`updateAum` enforces a maximum update rate through the cooldown check in line 502, but it never enforces a minimum freshness. There is no heartbeat and no maximum age on the reported price. The value `latestAum` is written by a single `aumUpdater` key, and once written it persists until the next report.

If the `aumUpdater` stops reporting, whether by key loss, operator failure, or neglect, the last `latestAum` stays in place forever. Every deposit in lines 258-287 and every `requestRedeem` in lines 291-320 keeps pricing against that stale value. Deposits mint shares, and redemptions lock payouts, at a price that no longer reflects the off-chain strategy. Because the strategy is private credit whose value drifts over time, a long gap between reports means users transact at a wrong price, and the first users to act on a stale-favorable price gain at the expense of the rest.

The architecture document states there is no on-chain freshness check, so this is an accepted limitation. We report it for completeness. We classify this issue as minor since it depends on the oracle stalling, but the contract gives users no protection while it is stale.

Recommendation

We recommend adding a maximum staleness bound. Store a maximum age and, in the price-sensitive paths `deposit` and `requestRedeem`, revert when `block.timestamp` is greater than `lastAumUpdatedAt` plus that age. This stops users from transacting against a price that has gone

stale, while leaving `claim` and the settlement paths open so funded redeemers are never trapped by a stale oracle.

Status: Resolved

3. The documented upgrade exit window is not enforceable on-chain

Severity: Minor

- `README.md:52-55`
- `contracts/NawaUSDTVault.sol:291-320`
- `contracts/NawaUSDTVault.sol:390-439`
- `contracts/NawaUSDTVault.sol:459-477`
- `contracts/NawaUSDTVault.sol:635-640`
- `contracts/NawaUSDTVault.sol:662-670`
- `scripts/deploy.js:26-35`

The `README.md` states that upgrades are gated by a 72-hour timelock, giving users a window to exit before an upgrade takes effect. The timelock delay itself is enforced, but the `NawaUSDTVault` contract does not provide a user-enforceable path to convert shares into USDT during that window.

In `contracts/NawaUSDTVault.sol:291-320`, `requestRedeem` only burns shares and creates a pending redemption request. It does not transfer USDT to the user:

```
function requestRedeem(uint256 shares) external nonReentrant whenNotPaused
returns (uint256 id) {
    if (shares < minRedeemShares) revert BelowMinimum();

    uint256 assetAmount = _convertToAssets(shares);
    if (assetAmount == 0) revert ZeroAmount();
    uint256 pps = _pricePerShare();

    _burn(msg.sender, shares);

    latestAum -= assetAmount;
    pendingRedemptionAum += assetAmount;
```

Actual settlement depends on operator-controlled or liquidity-dependent flows. `approveRequest` only enqueues the request for later funding. `approveAndPay` and `payApproved` require the `redeemManager`, `fundFromBackstop` requires the `backstop`, and `deposit netting` depends on later deposits.

The user escape hatch also does not provide a cash exit. In `contracts/NawaUSDTVault.sol:459-477`, `cancelRequest` only applies to Pending requests after `maxSettlementWindow` and returns shares rather than USDT:

```
function cancelRequest(uint256 id) external nonReentrant {
    RedemptionRequest storage r = _requests[id];
    if (r.user != msg.sender) revert NotAuthorized();
    if (r.status != Status.Pending) revert InvalidStatus();
    if (block.timestamp <= r.requestedAt + maxSettlementWindow) revert
    SettlementWindowNotElapsed();

    r.status = Status.Cancelled;
    r.outstanding = 0;
    latestAum += r.assetAmount;
    pendingRedemptionAum -= r.assetAmount;
    _mint(r.user, r.shares);
}
```

The default `maxSettlementWindow` is 120 days and can be configured up to 180 days, which is much longer than the documented 72-hour upgrade delay. In addition, `requestRedeem` is guarded by `whenNotPaused`; if the vault is paused during the upgrade delay, users cannot even initiate the redemption flow, and `unpause` is restricted to admin.

The deployment script also defaults the timelock controller to admin when `UPGRADE_CONTROLLER` is not set:

```
const upgradeController = env.UPGRADE_CONTROLLER || roles.admin;

const timelock = await Timelock.deploy(
    TIMELOCK_DELAY,
    [upgradeController],
    [upgradeController],
    ethers.constants.AddressZero
);
```

If the same party controls upgrade execution and redemption operations, the documented exit window becomes an operational assumption rather than an on-chain guarantee. Users can observe a queued upgrade, but they cannot force a USDT exit before execution.

Recommendation

We recommend updating the documentation to state that the 72-hour upgrade delay does not guarantee a user-enforceable exit, because redemptions are asynchronous and depend on approval, funding, and pause state.

If the exit-window guarantee is intended to be enforceable, implement a limited on-chain exit path that users can invoke during the upgrade delay. For example, allow users to redeem against idle on-chain liquidity without operator approval, with clear accounting rules and limits. If full

synchronous exits are impossible because assets are deployed off-chain, the documentation should state that limitation explicitly.

For mainnet deployments, require the upgrade controller to be independent from admin, redeemManager, and backstop, so upgrade execution and redemption operations are not controlled by the same party.

Status: Resolved

4. Zero update cooldown lets repeated oracle reports bypass the intended NAV movement rate

Severity: Minor

- contracts/NawaUSDVault.sol:499-528
- contracts/NawaUSDVault.sol:607-611

The NawaUSDVault contract uses `aumChangeLimitBps` to limit the price movement of each AUM update. This limit is measured against the current stored AUM on each call, so each accepted update becomes the baseline for the next one.

In `contracts/NawaUSDVault.sol:499-528`, `updateAum` enforces the per-update price movement cap by comparing the candidate price per share against the current price per share:

```
if (oldAum > 0) {
    uint256 oldPps = Math.mulDiv(oldAum + VIRTUAL_ASSETS, PPS_SCALE, supply + VIRTUAL_SHARES);
    uint256 candidatePps = Math.mulDiv(newAum + VIRTUAL_ASSETS, PPS_SCALE, supply + VIRTUAL_SHARES);
    if (_deviationBps(candidatePps, oldPps) > aumChangeLimitBps) revert ChangeLimitExceeded();
}
```

After the check passes, the report becomes the new baseline:

```
latestAum = newAum;
lastAumUpdatedAt = block.timestamp;
```

Because of this, repeated individually valid updates can compound. The `aumUpdater` can move the price by up to `aumChangeLimitBps` on one update, then move it by up to `aumChangeLimitBps` again on the next update, and so on.

This compounding is only rate-limited by `aumUpdateCooldownSecs`. However, in `contracts/NawaUSDTVault.sol:607-611`, `setAumUpdateCooldown` allows the admin to set `aumUpdateCooldownSecs` to 0 because it only enforces an upper bound:

```
function setAumUpdateCooldown(uint256 v) external onlyAdmin {
    if (v > MAX_AUM_UPDATE_COOLDOWN) revert ParameterOutOfRange();
    aumUpdateCooldownSecs = v;
    emit ParameterChanged("aumUpdateCooldownSecs", v);
}
```

In `updateAum`, the cooldown check only blocks an update when the current timestamp is lower than `lastAumUpdateAt + aumUpdateCooldownSecs`:

```
if (block.timestamp < lastAumUpdateAt + aumUpdateCooldownSecs) revert
CooldownActive();
```

When `aumUpdateCooldownSecs` is 0, repeated `updateAum` calls can succeed with the same `block.timestamp`. Each individual update can remain within `aumChangeLimitBps`, while the aggregate movement compounds across many updates without delay. For example, with the default 2% per-update limit, repeated valid updates can approximately double PPS after about 35 upward updates.

The default cooldown is 1 days, so this issue requires a non-default admin configuration and calls by the authorized `aumUpdater`. However, 0 is explicitly accepted by the setter, making this unsafe configuration available through normal parameter management. As a result, the per-update change limit no longer enforces a meaningful time-based NAV movement rate when cooldown is set to zero.

Recommendation

We recommend enforcing a nonzero minimum value for `aumUpdateCooldownSecs` so `aumChangeLimitBps` has a meaningful time component. The minimum should match the maximum NAV movement rate intended by the protocol.

For stronger protection, add a cumulative movement bound in addition to the per-update cap. For example, track total price-per-share movement over a rolling time window or fixed epoch and reject updates that would exceed that aggregate limit.

If immediate correction of mistaken reports is required, implement a separate correction path with explicit authorization, bounds, and events, rather than allowing ordinary reports to be stacked without delay.

Status: Acknowledged

The client acknowledges the issue stating that the zero lower bound on the cooldown is intentional, serving as an emergency correction mechanism for replacing mistaken oracle reports immediately. The trade-off is now documented in ARCHITECTURE.md, noting that a zero cooldown allows individually capped updates to compound without implying a meaningful time-based rate limit. Normal production operation enforces a nonzero cooldown, with zero reserved for incident response, and any parameter change emits a `ParameterChanged` event as an on-chain signal.

5. Loss-report front-running lets redeemers lock pre-loss NAV

Severity: Minor

- `contracts/NawaUSDTVault.sol:291-301`
- `contracts/NawaUSDTVault.sol:308-317`
- `contracts/NawaUSDTVault.sol:361-382`
- `contracts/NawaUSDTVault.sol:390-399`
- `contracts/NawaUSDTVault.sol:441-456`
- `contracts/NawaUSDTVault.sol:499-528`
- `contracts/NawaUSDTVault.sol:709-714`

The `NawaUSDTVault` contract prices redemption requests against the last reported `latestAum`. This request-time pricing is intentional, but it is unsafe when NAV updates are discrete, externally reported, and potentially observable or predictable.

In `contracts/NawaUSDTVault.sol:294-301`, `requestRedeem` calculates the redeemable asset amount from the current stored NAV, burns the user's shares, subtracts the amount from `latestAum`, and adds it to `pendingRedemptionAum`:

```
uint256 assetAmount = _convertToAssets(shares);
if (assetAmount == 0) revert ZeroAmount();
uint256 pps = _pricePerShare();

_burn(msg.sender, shares);

latestAum -= assetAmount;
pendingRedemptionAum += assetAmount;
```

The request stores that amount as the redemption liability:

```
_requests[id] = RedemptionRequest({
    user: msg.sender,
    shares: shares,
    assetAmount: assetAmount,
```

```
    outstanding: assetAmount,  
    originalPps: pps,  
    overridePps: 0,  
    requestedAt: block.timestamp,  
    status: Status.Pending  
});
```

Pending redemptions are not automatically repriced by later NAV updates. When `updateAum` reports a loss, it overwrites `latestAum` but does not adjust `pendingRedemptionAum` or the `assetAmount` and `outstanding` values stored in pending requests:

```
latestAum = newAum;  
lastAumUpdateAt = block.timestamp;
```

Because the requesting user's shares have already been burned, `totalSupply()` no longer includes those shares when the loss report is applied. The reported loss is therefore reflected only in `latestAum`, which backs the remaining share supply. The already requested redemption keeps its pre-loss payout, so the loss is concentrated on users who did not request redemption before the report.

A user who observes or predicts a negative NAV report can therefore call `requestRedeem` before the report and lock the pre-loss value. For example, assume Alice and Bob each hold 1,000 USDT of shares, and a default-allowed 2% loss on the full book is pending. If Bob requests redemption before the loss report, he locks a redemption amount of approximately 1,000 USDT. After the loss is reported, Alice's remaining shares are worth approximately 960 USDT, even though proportional loss allocation would leave both Alice and Bob with approximately 980 USDT.

The `redeemManager` has two limited tools to avoid or reduce this effect while a request is still Pending. It can reject the request, which returns the burned shares, or it can apply an override in `approveRequest`. However, both require active intervention before approval. The override is discretionary and capped by `overrideNavMaxDeviationBps`; in `contracts/NawaUSDTVault.sol:363`, overrides that exceed the configured deviation limit revert with `DeviationTooHigh`:

```
if (_deviationBps(overridePps, r.originalPps) > overrideNavMaxDeviationBps)  
    revert DeviationTooHigh();
```

The default override cap is 2%, and the configured cap cannot exceed 10%, so larger losses cannot be fully passed to the redeemer through the normal override path. Once a request is approved, the finalized payout remains fixed; later loss reports do not reprice it.

After approval, funding follows the approved queue. Early approved requests can therefore be funded before later requests, while still reflecting the pre-loss amount. This can worsen first-mover dynamics during a loss event, although the direct issue is the request-time locking of

value before the loss is reported.

The default 2% AUM change limit bounds the loss that can be applied in a single report. However, it does not remove the issue. The same mechanism can occur repeatedly across multiple negative reports or when a higher AUM change limit is configured.

Recommendation

We recommend using epoch-based settlement so redemption requests are queued and settled against the NAV for their settlement epoch, rather than permanently locking value at stale request-time NAV. Deposits and redemptions in the same epoch should be settled against a consistent NAV so users cannot exit immediately before a negative report and avoid losses already incurred by the vault.

If immediate request-time burns are retained, pending redemptions should remain repricable according to a clear NAV epoch policy until approval or funding. Another option is to settle each request from its burned share count at the price per share in effect at funding time.

If frozen request-time payouts are intended, document clearly that redeemers do not share post-request losses. In that case, size `overrideNavMaxDeviationBps`, funding policy, and operational loss-response procedures around the maximum loss the strategy can realistically incur before requests are approved or funded. Any exceptional repricing beyond the normal override cap should require stronger authorization, explicit bounds, clear evidence of the NAV event, and transparent events.

Status: Acknowledged

The client acknowledges the issue stating that request-time pricing is an intentional design choice and has been formally documented rather than changed. The ARCHITECTURE.md file now includes a "NAV timing" section that explicitly discloses the front-running risk and the bounds limiting its impact, including the per-update AUM change cap, a new price staleness bound, snapshot binding controls, and the redeem manager's ability to reject or override suspect requests. Epoch-based settlement has been noted as a potential future design direction.

6. Admin can retroactively delay the cancel deadline of already submitted redemption requests

Severity: Minor

- `contracts/NawaUSDVault.sol:635-641`
- `contracts/NawaUSDVault.sol:464-477`
- `contracts/NawaUSDVault.sol:697-701`

`cancelRequest` is the user escape hatch. It lets a requester reclaim a still Pending request once the settlement window has passed, so the user has recourse when the operator never approves or funds the request. The gate in line 468 is `block.timestamp <= r.requestedAt + maxSettlementWindow`. It reads the current `maxSettlementWindow` from storage, not the value that was in force when the request was made. The `isRequestOverdue` view in line 700 reads it the same way.

`setMaxSettlementWindow` in line 635 lets the admin change `maxSettlementWindow` at any time, up to `MAX_SETTLEMENT_WINDOW`, which is 180 days. The bound stops the admin from disabling the escape hatch outright, but it does not stop the change from reaching requests that already exist.

Because the gate uses the live value keyed on each request's fixed `requestedAt`, a change applies to in-flight requests. If a user submits a request while the window is 120 days, and the admin then raises it to 180 days before the first period elapses, the user must now wait 180 days from the original request time before `cancelRequest` succeeds. The only guaranteed exit moves further away after the fact, and the user never agreed to the new terms. The same live read also flips `isRequestOverdue`, so a request that was overdue can be made not overdue by a parameter change.

The impact is that the admin can extend the cancel deadline of requests that are already in flight, up to 180 days from the original request time, and can suppress the overdue signal for those requests. A user who is waiting to reclaim shares can be held longer than the terms that applied when the request was made. We classify this issue as minor since it needs the admin role and stays bounded to 180 days, but it changes the terms of a request after the user has committed to it.

Recommendation

We recommend snapshotting the settlement window into the request at request time, and reading that stored value in `cancelRequest` and `isRequestOverdue`. A later change to `maxSettlementWindow` then applies only to new requests and cannot move the deadline of a request that already exists. If a single global value must be kept, then allow a change to only shorten the effective window for existing requests, never to lengthen it.

Status: Resolved

7. Stale absolute AUM reports can misaccount post-snapshot state changes

Severity: Minor

- `contracts/NawaUSDTVault.sol:258-269`
- `contracts/NawaUSDTVault.sol:291-301`
- `contracts/NawaUSDTVault.sol:361-382`
- `contracts/NawaUSDTVault.sol:496-528`

The `NawaUSDTVault` contract stores `latestAum` as an absolute value. Deposits, redemption requests, and approval overrides update `latestAum` on-chain, but later oracle reports overwrite `latestAum` with the submitted `newAum`.

In `contracts/NawaUSDTVault.sol:258-269`, `deposit` mints shares and adds the deposited amount to `latestAum`:

```
asset.safeTransferFrom(msg.sender, address(this), amount);

// Price against pre-deposit state.
shares = _convertToShares(amount);
if (shares == 0) revert ZeroAmount();
if (totalSupply() + shares > mintingCap) revert MintingCapExceeded();

_mint(msg.sender, shares);
latestAum += amount;
```

In `contracts/NawaUSDTVault.sol:291-301`, `requestRedeem` burns shares, subtracts the locked asset amount from `latestAum`, and adds it to `pendingRedemptionAum`:

```
uint256 assetAmount = _convertToAssets(shares);
if (assetAmount == 0) revert ZeroAmount();
uint256 pps = _pricePerShare();
```

```
_burn(msg.sender, shares);

latestAum -= assetAmount;
pendingRedemptionAum += assetAmount;
```

In `contracts/NawaUSDTVault.sol:369-378`, `_applyOverride` can also move value between `latestAum` and `pendingRedemptionAum`:

```
if (newAmount > oldAmount) {
    uint256 delta = newAmount - oldAmount;
    if (delta > latestAum) revert AumUnderflow();
    latestAum -= delta;
    pendingRedemptionAum += delta;
} else if (newAmount < oldAmount) {
    uint256 delta = oldAmount - newAmount;
    latestAum += delta;
    pendingRedemptionAum -= delta;
}
```

The oracle report is expected to describe the current assets backing outstanding shares and to exclude `pendingRedemptionAum`:

```
/// @notice Report the off-chain assets backing outstanding shares, NET of any
///         performance fee and excluding pendingRedemptionAum. Fees are taken
///         off-chain before reporting; the contract charges no fee of its own.
function updateAum(uint256 newAum) external whenNotPaused {
```

However, `updateAum` does not bind a report to a snapshot block, timestamp, or accounting epoch. It overwrites `latestAum` with the absolute reported value:

```
latestAum = newAum;
lastAumUpdatedAt = block.timestamp;
```

If `newAum` was prepared from an off-chain snapshot, a deposit, redemption request, or approval override can be included after that snapshot but before the corresponding `updateAum` transaction. Unless the report is adjusted for those intervening on-chain actions, `updateAum` can misaccount the state.

For example:

1. At block `N`, the vault has `latestAum = 1,000 USDT` and `totalSupply = 1,000 shares`. The oracle prepares a report from that snapshot with `newAum = 1,000 USDT`.
2. At block `N + 1`, a user deposits `10 USDT`. The vault mints approximately `10 shares` and updates accounting to `latestAum = 1,010 USDT` and `totalSupply = 1,010 shares`.

3. At block $N + 2$, the oracle submits the stale `updateAum(1,000 USDT)` report. The report does not include the post-snapshot deposit, so `latestAum` is overwritten back to 1,000 USDT while `totalSupply` remains 1,010 shares.

The stored PPS falls from approximately 1.0 to $1,000 / 1,010 \approx 0.9901$. A later depositor can then mint shares at this depressed PPS. When a later corrected report includes the omitted 10 USDT, the depositor who entered at the depressed price captures part of that value from existing holders.

For redemption requests or upward approval overrides, the stale report can include assets that the contract has already moved into `pendingRedemptionAum`, double-counting value and inflating PPS. An inflated PPS makes later deposits mint fewer shares and later redemption requests lock more assets.

The AUM change-limit can reject large stale reports, but smaller stale reports can remain within the configured per-update price movement cap.

Recommendation

We recommend binding AUM reports to the on-chain state they are meant to describe.

Track the block number or timestamp of the latest NAV-affecting on-chain action, such as `deposit`, `requestRedeem`, and `_applyOverride`. Require each AUM report to declare its snapshot block or timestamp, and reject reports whose snapshot predates those state changes unless the reported `newAum` is explicitly adjusted for them.

Alternatively, use epoch-based settlement so deposits and redemption requests made after a snapshot do not affect live share supply or redemption liabilities until the next NAV is applied. This prevents stale absolute reports from omitting deposits or double-counting redemptions.

Status: Resolved

8. Empty-vault bootstrap deposits can be diluted by cancellation of a pending full redemption

Severity: Minor

- `contracts/NawaUSDTVault.sol:291-305`
- `contracts/NawaUSDTVault.sol:459-477`
- `contracts/NawaUSDTVault.sol:705-714`

The `NawaUSDTVault` contract allows a full redemption request to empty the live share supply while the redemption remains pending and cancellable. New deposits can then bootstrap the vault at a 1:1 price, even though the pending redemption can later be cancelled and reintroduced into the live share pool.

In `contracts/NawaUSDTVault.sol:291-305`, `requestRedeem` burns the requested shares, moves the locked asset amount from `latestAum` to `pendingRedemptionAum`, and clears `latestAum` when the burn reduces `totalSupply()` to zero:

```
uint256 assetAmount = _convertToAssets(shares);
if (assetAmount == 0) revert ZeroAmount();
uint256 pps = _pricePerShare();

_burn(msg.sender, shares);

latestAum -= assetAmount;
pendingRedemptionAum += assetAmount;

// When the last holder exits, clear the rounding dust so the vault
// re-bootstraps at pps 1.0 on the next deposit.
if (totalSupply() == 0) latestAum = 0;
```

After this, `totalSupply() == 0` and `latestAum == 0`, but `pendingRedemptionAum` may still be nonzero and the request status remains Pending.

Deposits during this interval are priced as if the vault were brand new. In `contracts/NawaUSDTVault.sol:705-707`, `_convertToShares` uses `latestAum + VIRTUAL_ASSETS` and `totalSupply() + VIRTUAL_SHARES`:

```
function _convertToShares(uint256 amount) private view returns (uint256) {
    return Math.mulDiv(amount, totalSupply() + VIRTUAL_SHARES, latestAum +
VIRTUAL_ASSETS);
}
```

With `totalSupply() == 0` and `latestAum == 0`, a depositor receives approximately one share unit per asset unit.

The original redeemer can later cancel the pending request after the settlement window. In `contracts/NawaUSDTVault.sol:459-477`, `cancelRequest` restores the request's asset amount to `latestAum`, subtracts it from `pendingRedemptionAum`, and mints the original shares:

```
r.status = Status.Cancelled;  
r.outstanding = 0;  
latestAum += r.assetAmount;  
pendingRedemptionAum -= r.assetAmount;  
_mint(r.user, r.shares);
```

If the original full-redemption request was made when PPS was below 1, the cancelling redeemer's restored shares were originally priced below the 1:1 bootstrap price paid by intervening depositors. When those shares and the locked asset amount are reintroduced, the cancelling redeemer receives a larger claim on the combined pool, while the intervening depositors are diluted.

This issue requires the full redeemer to own or redeem all live shares, the request to remain Pending until cancellation is allowed, and new deposits to enter before cancellation. If `redeemManager` approves or rejects the request before it becomes cancellable, this exact path is not available.

Recommendation

We recommend not allowing empty-vault bootstrap deposits while a cancellable full-supply redemption is still pending.

For example, make `deposit` revert when `totalSupply() == 0 && pendingRedemptionAum > 0`, unless a dedicated recovery path is active. Alternatively, prevent cancellation of a full-supply pending redemption after new shares have been minted, or design cancellation accounting so restored shares cannot dilute intervening depositors.

The simplest mitigation is to require `pendingRedemptionAum == 0` before the vault can bootstrap from `totalSupply() == 0`.

Status: Resolved

9. mintingCap does not remain a hard cap across pending redemptions

Severity: Minor

- contracts/NawaUSDTVault.sol:258-269
- contracts/NawaUSDTVault.sol:291-301
- contracts/NawaUSDTVault.sol:459-477
- contracts/NawaUSDTVault.sol:630-632

The NawaUSDTVault contract enforces mintingCap only during deposits. In contracts/NawaUSDTVault.sol:258-269, deposit checks the current live supply plus newly minted shares:

```
shares = _convertToShares(amount);
if (shares == 0) revert ZeroAmount();
if (totalSupply() + shares > mintingCap) revert MintingCapExceeded();

_mint(msg.sender, shares);
latestAum += amount;
```

When a user requests redemption, the shares are burned immediately. In contracts/NawaUSDTVault.sol:291-301, requestRedeem reduces totalSupply() before the redemption is approved or funded:

```
uint256 assetAmount = _convertToAssets(shares);
if (assetAmount == 0) revert ZeroAmount();
uint256 pps = _pricePerShare();

_burn(msg.sender, shares);

latestAum -= assetAmount;
pendingRedemptionAum += assetAmount;
```

This creates live supply headroom under mintingCap, even though the pending redemption may later be cancelled and reminted. Deposits can fill that headroom while the redemption request remains Pending.

After maxSettlementWindow elapses, the requester can cancel the pending request. In contracts/NawaUSDTVault.sol:459-477, cancelRequest remints the burned shares without checking mintingCap:

```
r.status = Status.Cancelled;  
r.outstanding = 0;  
latestAum += r.assetAmount;  
pendingRedemptionAum -= r.assetAmount;  
_mint(r.user, r.shares);
```

As a result, if deposits fill the cap room freed by the pending redemption, cancellation can push live `totalSupply()` above `mintingCap`. The request must remain Pending until cancellation is available; if it is approved or rejected before then, this exact path is not available.

This is a cap semantics issue rather than direct fund loss. The configured cap does not act as a hard invariant over total or future live share supply across the redemption request and cancellation lifecycle.

Recommendation

We recommend deciding whether `mintingCap` is intended to be a deposit-time cap or a hard live-supply cap.

If it is intended as a hard cap, include pending cancellable redemption shares in cap accounting so deposits cannot consume capacity that may need to be restored on cancellation. This should track pending shares, not pending asset value.

If it is only intended to limit new deposits at the time they execute, document that `mintingCap` is a deposit-time cap over current live supply only, and that pending redemption burns can temporarily free room which may later be restored through cancellation.

Status: Acknowledged

The client acknowledges the issue stating that `mintingCap` is intentionally a deposit-time cap over current live supply, not a hard invariant across the full redemption lifecycle. This design choice has been documented in commit 8c11bbd, clarifying that share burns on redemption requests legitimately free cap headroom, and that subsequent cancellations or rejections re-mint those shares without cap checks because they reverse an existing burn rather than admit new subscriptions. The client accepts that live supply may temporarily exceed `mintingCap` following such reversals.

10. User entry and exit functions lack slippage and deadline protection

Severity: Minor

- contracts/NawaUSDVault.sol:258-286
- contracts/NawaUSDVault.sol:291-320
- contracts/NawaUSDVault.sol:361-382
- contracts/NawaUSDVault.sol:499-528
- contracts/NawaUSDVault.sol:607-611
- contracts/NawaUSDVault.sol:679-684

The NawaUSDVault contract prices deposits and redemption requests against the stored NAV at the time the transaction is executed. Users can preview the expected result, but they cannot require the executed transaction to match that preview within a chosen tolerance.

In contracts/NawaUSDVault.sol:258-286, deposit accepts only amount and does not include a minSharesOut or deadline parameter:

```
function deposit(uint256 amount) external nonReentrant whenNotPaused returns
(uint256 shares) {
    if (amount < minDeposit) revert BelowMinimum();

    asset.safeTransferFrom(msg.sender, address(this), amount);

    // Price against pre-deposit state.
    shares = _convertToShares(amount);
    if (shares == 0) revert ZeroAmount();
    if (totalSupply() + shares > mintingCap) revert MintingCapExceeded();

    _mint(msg.sender, shares);
    latestAum += amount;
```

In contracts/NawaUSDVault.sol:291-320, requestRedeem accepts only shares and does not include a minAssetsOut or deadline parameter:

```
function requestRedeem(uint256 shares) external nonReentrant whenNotPaused
returns (uint256 id) {
    if (shares < minRedeemShares) revert BelowMinimum();

    uint256 assetAmount = _convertToAssets(shares);
    if (assetAmount == 0) revert ZeroAmount();
    uint256 pps = _pricePerShare();

    _burn(msg.sender, shares);
```

```
latestAum -= assetAmount;
pendingRedemptionAum += assetAmount;
```

The view functions in `contracts/NawaUSDTVault.sol:679-684` expose previews, but these previews do not bind execution:

```
function previewDeposit(uint256 amount) external view returns (uint256) {
    return _convertToShares(amount);
}

function previewRedeem(uint256 shares) external view returns (uint256) {
    return _convertToAssets(shares);
}
```

If `updateAum` is executed before the user's transaction, the transaction can execute at a different NAV than the user expected. A deposit can mint fewer shares after a positive NAV update, and a redemption request can lock fewer assets after a negative NAV update. Without a deadline, a delayed transaction can also execute much later at a materially different NAV.

The size of each ordinary oracle move is bounded by `aumChangeLimitBps`, but that bound can be configured up to 50%, and `aumUpdateCooldownSecs` can be set to 0. Therefore, users can experience materially worse-than-quoted execution from oracle timing even though the execution price is the current stored NAV when the transaction is processed.

For `requestRedeem`, a `minAssetsOut` check would only protect the amount locked at request time. The final payout can still be changed later if `redeemManager` applies an override through `_applyOverride`, so override policy and transparency should be considered separately.

Recommendation

We recommend adding slippage and deadline protection to user entry and exit functions.

For deposits, allow users to specify `minSharesOut` and revert if the actual minted shares are lower. For redemption requests, allow users to specify `minAssetsOut` and revert if the locked asset amount is lower. Both paths should accept a deadline and revert if `block.timestamp` exceeds it.

If backwards compatibility is required, keep the existing functions as convenience wrappers with no slippage protection, and add protected variants such as `depositWithMinShares(uint256 amount, uint256 minSharesOut, uint256 deadline)` and `requestRedeemWithMinAssets(uint256 shares, uint256 minAssetsOut, uint256 deadline)`. Document that the legacy variants accept any execution price.

Status: Resolved

11. Just-in-time deposits before positive NAV reports dilute existing holders

Severity: Minor

- `contracts/NawaUSDTVault.sol:258-269`
- `contracts/NawaUSDTVault.sol:499-528`
- `contracts/NawaUSDTVault.sol:705-714`

The `NawaUSDTVault` contract allows instant deposits to be priced against the last reported `latestAum`. This is unsafe because NAV updates are applied discretely through `updateAum`, while off-chain yield may accrue before it is reported on-chain.

In `contracts/NawaUSDTVault.sol:263-269`, `deposit` calculates shares from the current stored NAV, mints the shares, and only then adds the deposit amount to `latestAum`:

```
// Price against pre-deposit state.
shares = _convertToShares(amount);
if (shares == 0) revert ZeroAmount();
if (totalSupply() + shares > mintingCap) revert MintingCapExceeded();

_mint(msg.sender, shares);
latestAum += amount;
```

The share calculation uses the current `latestAum` and `totalSupply`:

```
function _convertToShares(uint256 amount) private view returns (uint256) {
    return Math.mulDiv(amount, totalSupply() + VIRTUAL_SHARES, latestAum +
VIRTUAL_ASSETS);
}
```

A user who observes or predicts a positive NAV report can deposit immediately before `updateAum`. The depositor receives shares at the stale pre-yield price. When the positive NAV report is applied, the previously accrued yield is spread across both existing holders and the just-in-time depositor, even though the depositor's capital did not earn that off-chain yield.

For example, assume Alice holds 1,000 USDT of shares and a +20 USDT NAV report is pending. If Bob deposits 1,000 USDT immediately before the report, Bob receives shares at the stale price. When the report is applied, Alice's shares are worth approximately 1,010 USDT instead of 1,020 USDT, while Bob's shares are worth approximately 1,010 USDT. Bob captures approximately half of Alice's already-accrued yield.

The default 2% AUM change limit bounds the value transfer from a single report, but it does not prevent the dilution. The issue can also repeat across multiple positive NAV reports, especially if reports are public in the mempool, scheduled, or otherwise predictable.

Recommendation

We recommend using epoch-based deposit settlement, or otherwise ensuring deposits made before a NAV update do not receive yield accrued before their entry.

For example, queue deposits and mint shares only at the next NAV report, or track pending deposits separately until the next report is applied. Deposits in an epoch should settle against a consistent NAV so users cannot enter immediately before a positive report and share in yield earned by earlier capital.

NAV freshness limits, deposit lockups, report-boundary fees, and less predictable reporting can reduce the attack surface, but they should be treated as secondary mitigations. They do not fully address the underlying stale-NAV pricing issue by themselves.

Status: Acknowledged

The client acknowledges the issue stating that the just-in-time deposit behaviour is an accepted consequence of request-time pricing, now documented alongside a related finding in commit 8c11bbd. Existing mitigations—the AUM change limit per report and the NAV staleness bound—are considered sufficient to constrain the value transferable in any single event. Operational policy of prompt, non-predictable reporting further reduces the attack surface. Epoch-based settlement is noted as a considered future design but is not part of the current version.

12. Redemption approval can change share pricing while the vault is paused

Severity: Minor

- `contracts/NawaUSDTVault.sol:390-400`
- `contracts/NawaUSDTVault.sol:407-420`
- `contracts/NawaUSDTVault.sol:361-383`

`pause` stops `deposit`, `requestRedeem`, and `updateAum`, since each carries the `whenNotPaused` modifier. `approveRequest` and `approveAndPay` do not carry `whenNotPaused`, so they stay callable while the vault is paused.

Both functions call `_applyOverride`. When an override price is supplied, `_applyOverride` moves value between `latestAum` and `pendingRedemptionAum` in lines 372-377. Moving `latestAum` changes the price per share for every holder. So while the vault is paused, the `redeemManager` can still shift the share price through an override, while users cannot deposit or redeem to react to it. This is an asymmetric state: pricing can move while users are frozen out.

The architecture document intentionally keeps `approveRequest` and `approveAndPay` callable during a pause so the redemption queue can wind down. That justification covers settlement and funding, but it does not address the override sub-path, which changes global share pricing. Winding the queue down does not require moving the price.

The impact is that during a pause the `redeemManager` can re-price shares within the override deviation cap while no user can exit at the old price. A downward override lowers the price per share for all holders, and an upward override raises one redeemer's payout at the other holders' expense. The move is bounded per request, but it happens at a time when users cannot respond. We classify this issue as minor since it needs the `redeemManager` role and the per-request move is bounded.

Recommendation

We recommend not allowing the override to move share pricing while the vault is paused. Either add `whenNotPaused` to the override path, or split approval so that the non-repricing part stays callable during a pause, to let the queue wind down, while the repricing override is blocked during a pause. Keep the settlement, funding, and claim paths open so redeemers are never trapped by a halt.

Status: Resolved

13. Stale funding queue slots can overstate the redemption backlog and delay surplus deployment

Severity: Minor

- `contracts/NawaUSDTVault.sol:271-286`
- `contracts/NawaUSDTVault.sol:350-354`
- `contracts/NawaUSDTVault.sol:422-438`
- `contracts/NawaUSDTVault.sol:691-694`
- `contracts/NawaUSDTVault.sol:731-733`
- `contracts/NawaUSDTVault.sol:738-768`

The `NawaUSDTVault` contract leaves funding queue slots in place when an approved request is settled directly through `payApproved`. In `contracts/NawaUSDTVault.sol:422-438`, `payApproved` marks the request as `Settled`, but does not remove its queue slot:

```
function payApproved(uint256 id) external onlyRedeemManager nonReentrant {
    RedemptionRequest storage r = _requests[id];
    if (r.status != Status.Approved) revert InvalidStatus();

    uint256 payout = r.outstanding;
    if (payout == 0) revert ZeroAmount();
    r.outstanding = 0;
    r.status = Status.Settled;
    pendingRedemptionAum -= payout;

    asset.safeTransferFrom(msg.sender, r.user, payout);
    emit RedemptionSettled(id, r.user, payout);
}
```

The queue later removes stale slots lazily. In `contracts/NawaUSDTVault.sol:745-750`, `_fundQueue` skips any queued request whose status is no longer `Approved`:

```
if (r.status != Status.Approved) {
    delete _fundingQueue[fundingHead];
    fundingHead++;
    continue;
}
```

Each stale slot consumes one `_fundQueue` iteration before it is removed. If enough stale slots accumulate at the head of the queue, `_fundQueue` can spend its entire `maxNettingIterations` budget skipping already settled entries and fund no live approved request behind them.

Until stale slots are skipped, queue-position checks also misrepresent the active backlog. In `contracts/NawaUSDTVault.sol:691-694`, `approvedUnfundedCount` returns `fundingTail - fundingHead`, and in `contracts/NawaUSDTVault.sol:731-733`, `_hasApprovedUnfunded` only checks whether `fundingHead < fundingTail`:

```
function approvedUnfundedCount() external view returns (uint256) {
    return fundingTail - fundingHead;
}

function _hasApprovedUnfunded() private view returns (bool) {
    return fundingHead < fundingTail;
}
```

These checks count stale queue slots even when the corresponding requests are already settled.

This can temporarily delay surplus deployment. During deposit, the vault first calls `_fundQueue` and then forwards surplus to `fundsManager` only if `_hasApprovedUnfunded` returns false:

```
uint256 consumed = _fundQueue(amount, maxNettingIterations);
uint256 surplus = amount - consumed;

if (surplus > 0 && !_hasApprovedUnfunded()) {
    if (fundsManager == address(0)) revert FundsManagerUnset();
    asset.safeTransfer(fundsManager, surplus);
} else {
    surplus = 0;
}
```

If stale slots exhaust `maxNettingIterations`, `_hasApprovedUnfunded` will still return true, so the deposit keeps the surplus idle instead of forwarding it to `fundsManager`. Real approved requests behind the stale slots also do not receive deposit netting until the stale slots are cleared.

The stale slots can later be cleared by another deposit or by `processQueue` if there is idle fundable balance. If there is no idle fundable balance, `processQueue` returns without clearing them. The funds are not lost, but deployment to the strategy is delayed, real queue progress can be delayed, and `approvedUnfundedCount` can overstate the actual approved-unfunded liability.

This issue can occur through normal use of `payApproved`; it does not require malicious behavior, but it does require the `redeemManager` role to create the stale settled slots.

Recommendation

We recommend making backlog accounting reflect active approved-unfunded liability rather than raw queue positions.

Prefer maintaining an $O(1)$ active approved-unfunded count or active outstanding liability amount. Increase it when requests are approved and decrease it whenever a request leaves the active approved-unfunded state, including full funding, direct settlement through `payApproved`, and any future approved-request cancellation flow. If tracking liability, decrease it when `_fundQueue` credits claimable, when `payApproved` settles outstanding amounts, and when any approved cancellation reverses outstanding liability.

Use the active count or liability in `_hasApprovedUnfunded` and `approvedUnfundedCount`, rather than `fundingTail - fundingHead`.

Alternatively, compact the queue when possible, such as removing or advancing stale head slots in `payApproved` when the paid request is at the queue head. Avoid unbounded queue scans in views or deposit paths. If `approvedUnfundedCount` is intended to report queue slots rather than active requests, rename or document it accordingly, and add a separate status-aware view for active outstanding approved requests.

Status: Resolved

14. Zero AUM reports cannot be recorded during empty-vault or total-loss states

Severity: Minor

- `contracts/NawaUSDTVault.sol:291-301`
- `contracts/NawaUSDTVault.sol:499-528`
- `contracts/NawaUSDTVault.sol:709-714`

The `NawaUSDTVault` contract rejects zero AUM reports before checking whether the vault has any shares outstanding. In `contracts/NawaUSDTVault.sol:499-514`, `updateAum` reverts when `newAum == 0` before it handles the empty-vault branch:

```
function updateAum(uint256 newAum) external whenNotPaused {
    if (msg.sender != aumUpdater) revert NotAuthorized();
    if (newAum == 0) revert ZeroAmount();
    if (block.timestamp < lastAumUpdatedAt + aumUpdateCooldownSecs) revert
    CooldownActive();

    uint256 supply = totalSupply();
    uint256 oldAum = latestAum;

    // An empty vault backs nothing: force AUM to zero regardless of the report,
    // so no update can price an arbitrary bootstrap and the next deposit starts
    // at pps 1.0. The change-limit is meaningless with no shares.
```

```

    if (supply == 0) {
        latestAum = 0;
        lastAumUpdatedAt = block.timestamp;
        emit AumUpdated(oldAum, 0, PPS_SCALE);
        return;
    }

```

When `totalSupply() == 0`, the correct AUM value in the contract's accounting model is zero. However, the oracle cannot call `updateAum(0)` to checkpoint that state. Instead, it must submit a nonzero dummy value, which the empty-vault branch then discards.

The same restriction also prevents exact reporting of a total loss while shares remain outstanding. If the true off-chain AUM falls to zero, `updateAum(0)` reverts and `latestAum` remains at its previous positive value unless the operator pauses the vault or uses another emergency procedure. The conversion functions continue to price against that stale value:

```

function _convertToAssets(uint256 shares) private view returns (uint256) {
    return Math.mulDiv(shares, latestAum + VIRTUAL_ASSETS, totalSupply() +
VIRTUAL_SHARES);
}

function _pricePerShare() private view returns (uint256) {
    return Math.mulDiv(latestAum + VIRTUAL_ASSETS, PPS_SCALE, totalSupply() +
VIRTUAL_SHARES);
}

```

While the vault is not paused, a holder can call `requestRedeem` and lock a redemption amount computed from the stale positive `latestAum`:

```

uint256 assetAmount = _convertToAssets(shares);
if (assetAmount == 0) revert ZeroAmount();
uint256 pps = _pricePerShare();

_burn(msg.sender, shares);

latestAum -= assetAmount;
pendingRedemptionAum += assetAmount;

```

The stale amount is not directly withdrawable without later funding, but it creates a redemption liability that can exceed the actual backing after a true total loss. This complicates incident handling and can make fair settlement harder unless the vault is paused or placed into a defined recovery mode before users interact.

Recommendation

We recommend allowing zero AUM reports when `totalSupply() == 0`. For example, move the `empty-vault` branch before the `newAum == 0` check, or make the zero-AUM check conditional on `totalSupply() > 0`.

If exact total-loss reporting should be supported while shares remain outstanding, implement it as an explicit recovery mode with clear pricing semantics and safeguards. During that mode, normal deposits and redemption requests should not continue unchanged. In particular, prevent users from creating new redemption liabilities from stale positive `latestAum` after a zero-AUM event is known, and define how existing shares and pending redemptions are settled.

Status: Resolved

15. Initialization and approval paths emit incomplete events, leaving off-chain trackers with gaps

Severity: Informational

- `contracts/NawaUSDVault.sol:203-241`
- `contracts/NawaUSDVault.sol:132`
- `contracts/NawaUSDVault.sol:390-420`
- `contracts/NawaUSDVault.sol:577-647`

Four related gaps make the event log an incomplete record of state changes.

First, `initialize` in lines 203-241 sets every role and every default parameter but emits no event. Each role has a matching `...Changed` event, and each parameter has a `ParameterChanged` event, but those only fire on later updates. The initial assignment is silent. An indexer that rebuilds role and parameter history from logs cannot see the first value of any role or parameter, only the first change.

Second, the `RedemptionApproved` event in line 132 has no `user` field, neither indexed nor plain. Every other redemption lifecycle event carries `address indexed user`. So a consumer cannot filter approvals by user, and cannot even read the user from the approval log. It must join back to `RedemptionRequested` by `id`.

Third, `approveAndPay` in lines 407-420 applies a price override through `_applyOverride`, exactly like `approveRequest`, but it emits only `RedemptionSettled`. It never emits `RedemptionApproved`, so the `overridePps` it applied is not logged anywhere. `approveRequest` in line 399 does log that value. So the same re-price is observable on one approval path and invisible on the other.

Fourth, the role setters such as `setRedeemManager`, `setAumUpdater`, `setPauser`, and `setBackstop`, and the parameter setters, emit their `...Changed` and `ParameterChanged` events without checking whether the new value differs from the current one. Setting a role or a parameter to the value it already holds still succeeds and still emits an event. An off-chain consumer then sees a change event for a state that did not change, which misleads any monitor that counts or reacts to these events.

The impact is that off-chain systems that rely on events, such as indexers, dashboards, monitors, and accounting tools, get an incomplete or misleading picture: no initial configuration, approvals that cannot be attributed to a user, overrides applied on the direct settle path that leave no trace, and change events emitted for no-op writes. No on-chain behavior or funds are affected. We classify this issue as informational since it concerns observability and not on-chain security.

Recommendation

We recommend emitting the initial role and parameter values from `initialize`, either by reusing the existing `...Changed` and `ParameterChanged` events or by adding a single initialization event. Add `address indexed user` to `RedemptionApproved`. Emit `RedemptionApproved` inside `approveAndPay` as well, so the applied `overridePps` is logged on both approval paths. In the role and parameter setters, load the current value first and skip the write and the event, or revert, when the new value equals the current one, so an event is emitted only for a real change.

Status: Resolved

16. No mechanism to burn or freeze wrongly minted shares, upgrade is the only remedy

Severity: Informational

- `contracts/NawaUSDTVault.sol:258-287`
- `contracts/NawaUSDTVault.sol:268`
- `contracts/NawaUSDTVault.sol:298`
- `contracts/NawaUSDTVault.sol:454`
- `contracts/NawaUSDTVault.sol:474`

The share token is a standard `ERC20Upgradeable`. Shares are only created through `_mint` in `deposit` in line 268, in `rejectRequest` in line 454, and in `cancelRequest` in line 474. Shares are only removed through `_burn` in `requestRedeem` in line 298. There is no admin function to burn, freeze, or claw back shares held by a specific address.

This means that if shares are ever created or credited in error, for any reason, the contract has no on-chain way to remove them or to stop the holder from using them. The dilution to every other holder stays in place.

pause does not solve this. It only stops deposit, requestRedeem, and updateAum. It does not touch existing balances. Share transfers are not paused, so the shares can still move between accounts. Claims and direct settlements of requests that are already approved also stay callable while paused. So a pause limits new deposits and new redemption requests, but it does not contain shares that already exist.

The only permanent remedy is a proxy upgrade. That path is gated to the upgrader timelock, so it is announced on-chain and delayed before it can take effect. It cannot be used as a fast response. We classify this issue as informational since it describes a recovery limitation and not an exploitable flaw on its own. It matters because it removes the last line of defense if any other bug, an oracle mistake, or an operational error ever mints shares that should not exist.

Recommendation

We recommend deciding up front how the protocol would recover from wrongly minted shares, and documenting that plan. If an on-chain remedy is wanted, add a narrow, access-controlled function that can burn shares from a named address, guarded by the same timelock or multisig that controls upgrades, and emit a clear event when it is used. If the team prefers to rely on upgrades only, then state that in the trust assumptions, so the timelock delay is understood as the accepted recovery time.

Status: Resolved

17. Extreme rounded-to-zero price can brick ordinary AUM updates

Severity: Informational

- contracts/NawaUSDTVault.sol:517-523
- contracts/NawaUSDTVault.sol:717-721

The NawaUSDTVault contract checks AUM update deviation by comparing the candidate price per share against the previous price per share. In contracts/NawaUSDTVault.sol:517-523, updateAum computes oldPps and passes it as the denominator to _deviationBps:

```
if (oldAum > 0) {
    uint256 oldPps = Math.mulDiv(oldAum + VIRTUAL_ASSETS, PPS_SCALE, supply +
VIRTUAL_SHARES);
    uint256 candidatePps = Math.mulDiv(newAum + VIRTUAL_ASSETS, PPS_SCALE,
```

```
supply + VIRTUAL_SHARES);
    if (_deviationBps(candidatePps, oldPps) > aumChangeLimitBps) revert
    ChangeLimitExceeded();
}
```

In contracts/NawaUSDTVault.sol:717-721, `_deviationBps` returns `type(uint256).max` when the denominator is zero:

```
function _deviationBps(uint256 a, uint256 b) private pure returns (uint256) {
    if (b == 0) return type(uint256).max;
    uint256 diff = a > b ? a - b : b - a;
    return (diff * BPS_DENOMINATOR) / b;
}
```

If `oldAum > 0` but `oldPps` floors to zero, every ordinary `updateAum` call reverts with `ChangeLimitExceeded`. This requires:

```
(oldAum + VIRTUAL_ASSETS) * PPS_SCALE < totalSupply() + VIRTUAL_SHARES
```

Normal deposit and redemption flows keep share supply and AUM approximately proportional, so this condition is not expected to occur in ordinary operation. It is primarily a latent liveness cliff if another bug or recovery path creates an extremely large live share supply relative to AUM.

Recommendation

We recommend enforcing invariants that prevent near-zero AUM with inflated live share supply. In particular, prevent states where `latestAum == 0 && totalSupply() > 0` can be used to mint excessive shares or otherwise create an extreme supply-to-AUM ratio.

Additionally, handle the `oldPps == 0` case explicitly inside `updateAum`. After computing `oldPps`, do not pass it into `_deviationBps` as an ordinary denominator. Instead, route this state to a dedicated recovery branch, for example:

```
uint256 oldPps = Math.mulDiv(oldAum + VIRTUAL_ASSETS, PPS_SCALE, supply +
VIRTUAL_SHARES);
uint256 candidatePps = Math.mulDiv(newAum + VIRTUAL_ASSETS, PPS_SCALE, supply +
VIRTUAL_SHARES);

if (oldPps == 0) {
    // Enter a governed, bounded recovery path instead of ordinary deviation
    logic.
    revert RecoveryRequired();
}

if (_deviationBps(candidatePps, oldPps) > aumChangeLimitBps) revert
ChangeLimitExceeded();
```

The recovery path should be explicit, authorized, bounded, and emit clear events. For example, it can require governance or timelock authorization and cap the first nonzero recovery PPS to a documented maximum. Do not simply treat `oldPps == 0` as zero deviation or skip the comparison in ordinary `updateAum`, because that would allow unbounded price movement from a degenerate state.

Status: Resolved

18. BackstopFunded reports a consumed amount that can exceed the backstop transfer

Severity: Informational

- `contracts/NawaUSDTVault.sol:484-492`
- `contracts/NawaUSDTVault.sol:138`

`fundFromBackstop` pulls amount USDT from the backstop, then funds the queue with `_fundQueue(_idleFundable(), maxIterations)`. The budget it passes is `_idleFundable()`, which is the whole idle balance of the contract, not just amount. The returned consumed is then emitted in `BackstopFunded(msg.sender, amount, consumed)` in line 491.

When the contract already holds idle USDT before the call, for example surplus that a prior deposit kept in the contract, `consumed` can be larger than `amount`. The event then pairs the backstop's own contribution, `amount`, with a `consumed` that includes USDT the backstop did not send in this call. An off-chain tool that sums `consumed` to measure how much the backstop has funded will over-count, and a tool that assumes `consumed <= amount` will be wrong.

We classify this issue as informational since the on-chain accounting stays correct, but the event can corrupt off-chain accounting that is built on it.

Recommendation

We recommend making the event unambiguous. The preferred option is to keep funding from the full idle balance, since that is good for liveness, but change the event so it reports the backstop transfer and the funded total as clearly separate values, and if useful the pre-existing idle that was also consumed. If a simpler event is preferred, cap the funding budget at `amount` so that `consumed` can never exceed the backstop transfer, at the cost of leaving any pre-existing idle for a later `processQueue` call.

Status: Resolved

19. Non-contiguous storage layout increases upgrade maintenance risk

Severity: Informational

- contracts/NawaUSDVault.sol:82-126
- contracts/NawaUSDVault.sol:182-196
- contracts/NawaUSDVault.sol:247-250
- contracts/NawaUSDVault.sol:770-771
- scripts/deploy.js:53-60
- scripts/upgrade.js:15-24

The NawaUSDVault contract is upgradeable and relies on storage layout compatibility across implementations. Most contract-defined state variables are grouped near the top of the contract, but asset is declared later after the InitParams struct.

In contracts/NawaUSDVault.sol:182-196, asset is declared separately from the other role and accounting state:

```
struct InitParams {
    IERC20 asset;
    address admin;
    address fundsManagerAssigner;
    address fundsManager;
    address redeemManager;
    address aumUpdater;
    address pauser;
    address backstop;
    address upgrader;
}

// Set once in initialize. Treat as upgrade-immutable: a future upgrade must
// never add a setter, as repointing the asset would corrupt all accounting.
IERC20 public asset;
```

The storage gap remains at the end of the contract:

```
/// @dev Reserved storage for future versions; preserves layout across upgrades.
uint256[50] private __gap;
```

The current storage layout is valid. However, storage slots are assigned in source declaration order. If a future developer adds a new state variable near the earlier state-variable section, before the existing asset declaration, the new variable would take the storage slot currently used by asset. The asset variable would shift into the next slot, corrupting the live proxy's storage layout.

This is especially risky because `asset` backs all token transfers and accounting assumptions, but it is not visually grouped with the other state variables. The non-contiguous placement makes an unsafe insertion easier to miss during future maintenance.

The deployment and upgrade scripts use OpenZeppelin upgrade validation and explicitly rely on the preserved `.openzeppelin` manifest. This should catch incompatible layout changes as long as the original deployment manifest is preserved and used for future upgrades. If the manifest is lost, ignored, or recreated from the wrong source, this protection can be weakened.

There is no on-chain storage layout validation. In `contracts/NawaUSDTVault.sol:247-250`, `_authorizeUpgrade` only checks that the caller is upgrader:

```
function _authorizeUpgrade(address) internal view override {  
    if (msg.sender != upgrader) revert NotAuthorized();  
}
```

This is standard for UUPS upgradeability, but it means layout compatibility and downgrade prevention depend on the off-chain upgrade process rather than an on-chain check.

Recommendation

We recommend that, before mainnet deployment or before the storage layout is otherwise frozen, move `asset` next to the other state variables so all contract-defined storage is contiguous and easier to review.

After deployment, never reorder existing state variables or insert new variables before existing ones, including `asset`. Preserve the original `.openzeppelin/<network>.json` manifest and validate every upgrade against that original manifest. Avoid using `forceImport` with a mutated working copy as the baseline for future upgrades.

Continue requiring `prepareUpgrade` layout validation for every upgrade. If the project also wants downgrade protection, consider adding an explicit implementation version and requiring future upgrades to move to a strictly higher version. This does not replace storage layout validation, but it can reduce operational risk from accidentally applying an older implementation.

Status: Resolved

20. No emergency stop can halt outflows if funds are ever credited in error

Severity: Informational

- `contracts/NawaUSDTVault.sol:258`
- `contracts/NawaUSDTVault.sol:291`
- `contracts/NawaUSDTVault.sol:499`
- `contracts/NawaUSDTVault.sol:324-334`
- `contracts/NawaUSDTVault.sol:350-353`
- `contracts/NawaUSDTVault.sol:484-492`

Only three functions carry `whenNotPaused`: `deposit` in line 258, `requestRedeem` in line 291, and `updateAum` in line 499. Everything downstream of a redemption request stays callable while paused. This includes `approveRequest`, `processQueue`, `fundFromBackstop`, `payApproved`, `approveAndPay`, and the exit functions `claim` and `claimFor`.

This is a deliberate choice. It lets the operator run a soft halt that stops new deposits, new requests, and new oracle reports, while redeemers still exit and the backstop still winds the queue down. A halt does not trap funded redeemers.

The gap is that this same choice leaves the protocol with no way to stop an outflow if `claimable` is ever credited in error. Such an error can come from a math mistake in the conversion or funding code, or from a bad oracle report. Once an amount sits in `claimable`, the only thing before it leaves the contract is `claim`, and `claim` performs a real USDT transfer that cannot be reversed. Pausing does not help, because the whole path from `approve` to `fund` to `claim` stays open.

The two cases differ in blast radius. A bad oracle report can be stopped from getting worse, since `updateAum` is pausable, and each report is capped by `aumChangeLimitBps`, so the damage per report is bounded. However, a report that is already applied can still flow out through the unpausable path. A math bug has no cap and no brake at all.

The only recovery is a proxy upgrade. That path is gated to the upgrader timelock, so it is announced and delayed. It cannot beat an attacker who drains `claim` in real time. We classify this issue as informational since it only causes harm if another bug or a bad oracle report first credits `claimable` in error, but it removes the last line of defense in exactly that case.

Recommendation

We recommend adding a separate emergency halt, controlled by a guardian multisig, that can freeze the exit and funding paths, meaning `claim`, `claimFor`, `processQueue`, `fundFromBackstop`, `payApproved`, and `approveAndPay`. Keep it distinct from the normal pause, so the soft halt wind-down mode still works and honest redeemers are not trapped during routine pauses.

Make the emergency halt time-bounded and auto-expiring, so it cannot become a permanent freeze of user funds. A short, self-lifting freeze gives the protocol a real circuit breaker on the final withdrawal, without handing any role a permanent power to trap funds.

Status: Resolved

21. Initialization does not enforce role separation, so one key can hold every privileged role

Severity: Informational

- `contracts/NawaUSDVault.sol:203-241`

The `initialize` function in lines 203-241 accepts `admin`, `fundsManagerAssigner`, `fundsManager`, `redeemManager`, `aumUpdater`, `pauser`, `backstop`, and `upgrader` as independent parameters. It checks that each required role is not the zero address in lines 204-213, but it never checks that the roles are held by distinct addresses.

So a deployment can set the same address for several or all of these roles, which removes the separation of privileges. If that single key is compromised, the holder gains control of every privileged operation at once.

This matters more than a generic key-management note, because the design relies on specific separations that the contract enforces elsewhere but not at initialization:

- The architecture states that the `admin` cannot change where deposits are sent, since `setFundsManager` is callable only by `fundsManagerAssigner`. If `initialize` is called with `admin` equal to `fundsManagerAssigner`, that separation is defeated from the start, and the `admin` can repoint the deposit destination.
- Upgrades are meant to be gated to a separate `upgrader`, expected to be a timelock. If `admin` equals `upgrader`, the `admin` can upgrade the implementation directly, bypassing the intended delay.
- Several roles combine into value-extraction paths. If `aumUpdater` and `redeemManager` are the same key, the holder can pair a redemption approval, rejection, or price override with a chosen AUM report and move value between shareholders and a favored redeemer within the per-update change limit.

We classify this issue as informational since it depends on the deployment configuration, but the contract does not prevent, or even warn against, a configuration that removes the separations the design depends on.

Recommendation

We recommend adding validation in `initialize` that rejects a configuration in which roles that must stay independent share an address. At a minimum, require `admin`, `fundsManagerAssigner`, and `upgrader` to be distinct, since the design already separates those on-chain, and require `aumUpdater` and `redeemManager` to be distinct. If sharing is acceptable for some roles that are held by multisigs, document the exact expected deployment configuration and state which roles may and may not coincide.

Status: Resolved

22. NAV guardrails are admin-governed operational controls

Severity: Informational

- `contracts/NawaUSDTVault.sol:119-123`
- `contracts/NawaUSDTVault.sol:230-234`
- `contracts/NawaUSDTVault.sol:583-617`
- `ARCHITECTURE.md:209-224`
- `README.md:39-48`

The `NawaUSDTVault` contract exposes several NAV-related guardrails: `aumChangeLimitBps`, `aumUpdateCooldownSecs`, and `overrideNavMaxDeviationBps`. These parameters constrain ordinary oracle updates and redemption overrides, but they are governed by `admin`.

In `contracts/NawaUSDTVault.sol:601-617`, `admin` can change the AUM update limit, AUM update cooldown, and override deviation cap:

```
function setAumChangeLimitBps(uint256 v) external onlyAdmin {
    if (v == 0 || v > MAX_AUM_CHANGE_LIMIT_BPS) revert ParameterOutOfRange();
    aumChangeLimitBps = v;
    emit ParameterChanged("aumChangeLimitBps", v);
}

function setAumUpdateCooldown(uint256 v) external onlyAdmin {
    if (v > MAX_AUM_UPDATE_COOLDOWN) revert ParameterOutOfRange();
    aumUpdateCooldownSecs = v;
    emit ParameterChanged("aumUpdateCooldownSecs", v);
}

function setOverrideNavMaxDeviationBps(uint256 v) external onlyAdmin {
    if (v > OVERRIDE_NAV_DEVIATION_CEILING_BPS) revert ParameterOutOfRange();
```

```
    overrideNavMaxDeviationBps = v;  
    emit ParameterChanged("overrideNavMaxDeviationBps", v);  
}
```

admin can also rotate the AUM reporting role. In `contracts/NawaUSDVault.sol:583-586`, `setAumUpdater` allows admin to replace `aumUpdater`:

```
function setAumUpdater(address a) external onlyAdmin {  
    if (a == address(0)) revert ZeroAddress();  
    emit AumUpdaterChanged(aumUpdater, a);  
    aumUpdater = a;  
}
```

The parameter bounds still limit what admin can configure. For example, `aumChangeLimitBps` is capped at 50%, `overrideNavMaxDeviationBps` is capped at 10%, and `aumUpdateCooldownSecs` is capped at 7 days. However, admin can loosen these guardrails within those bounds, including setting the AUM update cooldown to 0.

The documentation already discloses that admin controls configuration and limits, and `ARCHITECTURE.md` lists the relevant parameter bounds. The key trust-model point is that these NAV guardrails are operational controls governed by admin, not independent protections against admin.

Recommendation

We recommend documenting the trust model clearly: admin is price-sensitive governance because it can affect NAV reporting and redemption economics through parameter changes and role rotation.

If stronger assurances are desired, consider hardening NAV-sensitive actions with additional controls. This can include placing `aumUpdater` rotation, `aumChangeLimitBps` changes, `aumUpdateCooldownSecs` changes, and `overrideNavMaxDeviationBps` changes behind a timelock or multisig, adding delayed activation for pricing-sensitive parameter changes, and separating routine administration from emergency pricing authority.

Status: Acknowledged

The client acknowledges the issue stating that the trust model has been clarified in the documentation to explicitly recognise admin as price-sensitive governance rather than a neutral operational role. The updated `ARCHITECTURE.md` now discloses that the NAV guardrails constrain the oracle on the admin's behalf, not the admin itself, and that admin can loosen these parameters within hard-coded bounds. The recommendation to hold admin in a multisig and optionally place NAV-sensitive setters behind a timelock has also been incorporated into the documentation.

23. Deploy default can assign upgrade timelock control to admin

Severity: Informational

- `scripts/deploy.js:26-35`
- `contracts/NawaUSDVault.sol:247-250`

The `NawaUSDVault` contract correctly restricts UUPS upgrades to the configured upgrader. In `contracts/NawaUSDVault.sol:247-250`, `_authorizeUpgrade` only allows calls from upgrader:

```
function _authorizeUpgrade(address) internal view override {  
    if (msg.sender != upgrader) revert NotAuthorized();  
}
```

The deploy script initializes upgrader as a `TimelockController`, which enforces a 72-hour delay. However, the party that controls the timelock defaults to the same address as `admin` if `UPGRADE_CONTROLLER` is not set.

In `scripts/deploy.js:26-35`, `upgradeController` is used as both proposer and executor for the timelock:

```
// Whoever controls the upgrade timelock (proposer + executor). Defaults to  
admin.  
const upgradeController = env.UPGRADE_CONTROLLER || roles.admin;  
  
// 1. Timelock that will hold the upgrade right (enforces the 72h delay).  
const Timelock = await ethers.getContractFactory("TimelockController");  
const timelock = await Timelock.deploy(  
    TIMELOCK_DELAY,  
    [upgradeController],  
    [upgradeController],  
    ethers.constants.AddressZero // no separate timelock-admin role  
);
```

As a result, if `UPGRADE_CONTROLLER` is omitted during deployment, the same day-to-day admin entity can also schedule and execute upgrades after the timelock delay. The contract-level upgrade gate remains intact, but the intended separation between routine administration and upgrade governance can collapse to a single controller plus a timer.

No distinct timelock admin is configured at deployment, so the timelock governance model depends entirely on the initial role configuration.

Recommendation

We recommend that for mainnet deployments, `UPGRADE_CONTROLLER` is required to be explicitly set to an independent multisig or governance address distinct from `ADMIN_ADDRESS`. The deploy script should fail when `UPGRADE_CONTROLLER` is unset or equals `ADMIN_ADDRESS` on mainnet.

For non-mainnet deployments, emit a clear warning when the upgrade controller defaults to admin, and document that this is a testing convenience rather than a production governance model.

Status: Resolved

24. Configured asset is not validated against USDT assumptions

Severity: Informational

- `contracts/NawaUSDTVault.sol:182-196`
- `contracts/NawaUSDTVault.sol:203-245`
- `scripts/deploy.js:3-21`

The `NawaUSDTVault` contract is branded and parameterized as a USDT vault. It hardcodes the share token decimals to 6, and asset-denominated accounting such as `latestAum`, redemption amounts, and `minDeposit` assumes USDT-like units.

In `contracts/NawaUSDTVault.sol:243-245`, the share token always reports 6 decimals:

```
function decimals() public pure override returns (uint8) {
    return SHARE_DECIMALS;
}
```

The initializer accepts the asset address through `InitParams`. In `contracts/NawaUSDTVault.sol:203-220`, `initialize` only verifies that the asset address is nonzero before storing it:

```
function initialize(InitParams memory p) external initializer {
    if (
        address(p.asset) == address(0) ||
        p.admin == address(0) ||
        p.fundsManagerAssigner == address(0) ||
        p.redeemManager == address(0) ||
        p.aumUpdater == address(0) ||
        p.pauser == address(0) ||
        p.backstop == address(0) ||
        p.upgrader == address(0)
    ) {
        revert();
    }
}
```

```

    ) revert ZeroAddress();

    __ERC20_init("Nawa USDT Vault", "nzUSDT");
    __ReentrancyGuard_init();
    __Pausable_init();
    __UUPSUpgradeable_init();

    asset = p.asset;

```

The default minimum deposit is also expressed using the same 6 decimal scale:

```

minDeposit = 100 * (10 ** SHARE_DECIMALS); // 100 USDT

```

The mainnet deploy script defaults to Ethereum USDT, but it also allows `ASSET_ADDRESS` to override the asset:

```

// USDT mainnet address. Override via env for testnets / mocks.
const MAINNET_USDT = "0xdAC17F958D2ee523a2206206994597C13D831ec7";
// ...
asset: env.ASSET_ADDRESS || MAINNET_USDT,

```

If a wrong asset address or a token with incompatible decimals is configured at deployment, the vault's minimum amounts, AUM units, redemption amounts, and user-facing accounting can be incorrect. The asset is intended to be immutable after initialization, so a deployment mistake would require an upgrade or migration to correct.

Recommendation

We recommend for mainnet deployments, validate the configured asset in the deployment script or deployment checklist. The script should fail if `ASSET_ADDRESS` is set to an address other than Ethereum mainnet USDT unless an explicit unsafe override flag is provided.

If the vault is intended to support only Ethereum USDT, keep the contract and deployment process specialized to that asset and document the assumption clearly. If generic asset support is desired, update the design more broadly: derive or configure asset decimals, minimum amounts, AUM reporting units, token naming, and documentation consistently for the configured asset. Off-chain deployment validation may be preferable to on-chain metadata checks because not all ERC20 tokens implement metadata consistently.

Status: Resolved

25. Discretionary redemption overrides can shift value between redeemers and remaining holders

Severity: Informational

- `contracts/NawaUSDTVault.sol:131-132`
- `contracts/NawaUSDTVault.sol:361-382`
- `contracts/NawaUSDTVault.sol:390-399`
- `contracts/NawaUSDTVault.sol:407-419`
- `contracts/NawaUSDTVault.sol:687-689`

The `NawaUSDTVault` contract allows `redeemManager` to override a redemption request's price at approval time. The override is bounded by `overrideNavMaxDeviationBps`, but the contract does not require the override to reference a fresh oracle NAV, a settlement epoch, or another on-chain justification.

In `contracts/NawaUSDTVault.sol:361-382`, `_applyOverride` accepts any nonzero `overridePps` within the configured deviation from the request-time `originalPps`:

```
function _applyOverride(RedemptionRequest storage r, uint256 overridePps)
private {
    if (overridePps == 0 || overridePps == r.originalPps) return;
    if (_deviationBps(overridePps, r.originalPps) > overrideNavMaxDeviationBps)
revert DeviationTooHigh();

    uint256 newAmount = Math.mulDiv(r.shares, overridePps, PPS_SCALE);
    if (newAmount == 0) revert ZeroAmount();

    uint256 oldAmount = r.assetAmount;
    if (newAmount > oldAmount) {
        uint256 delta = newAmount - oldAmount;
        if (delta > latestAum) revert AumUnderflow();
        latestAum -= delta;
        pendingRedemptionAum += delta;
    } else if (newAmount < oldAmount) {
        uint256 delta = oldAmount - newAmount;
        latestAum += delta;
        pendingRedemptionAum -= delta;
    }

    r.assetAmount = newAmount;
    r.outstanding = newAmount;
    r.overridePps = overridePps;
}
```

A lower override reduces the redeemer's payout and moves the difference back into `latestAum`, benefiting remaining holders. A higher override increases the redeemer's payout and removes the difference from `latestAum`, reducing the value backing remaining holders. The value shift is bounded by `overrideNavMaxDeviationBps`, which defaults to 2%.

The override can be applied through both `approveRequest` and `approveAndPay`. In each case, the only on-chain constraints are the deviation cap and the `redeemManager` authorization.

The `RedemptionApproved` event emits the final `assetAmount` and `overridePps`, but it does not emit the request-time `originalPps` or the resulting deviation. The `originalPps` remains available through `getRequest`, so this is primarily an accountability and monitoring limitation rather than missing state.

Recommendation

We recommend adding clearer accountability or constraints around redemption overrides.

Emit enough information for users and monitors to evaluate each override directly from events, such as the request-time `originalPps`, the applied `overridePps`, and the deviation. Consider requiring overrides to follow a documented settlement-epoch policy or reference a fresh NAV report.

If discretionary overrides remain necessary for operations, lower the default override cap where feasible and require stronger authorization, such as multisig approval, for overrides above a small threshold.

Status: Resolved

26. Admin handover does not rotate operator roles, leaving stale powers such as pause

Severity: Informational

- `contracts/NawaUSDVault.sol:649-660`
- `contracts/NawaUSDVault.sol:664-671`

`transferAdmin` and `acceptAdmin` move only the `admin` role. They do not touch the other roles that the admin assigns, which are `pauser`, `redeemManager`, `aumUpdater`, and `backstop`.

The `pause` function is callable by `pauser` or `admin` in line 665. The `unpause` function is `onlyAdmin` in line 669. So after `acceptAdmin` sets `admin` to the new address in line 658, the new admin can always unpause, and the old admin loses pause power through the `admin` role. There is no state where the contract is stuck paused with a locked-out admin.

The real gap is the `pauser` role. It is separate, and the handover does not rotate it. If the outgoing operator also holds `pauser`, then after the admin handover the old operator keeps the power to pause. The old operator can pause again after each unpause. The new admin can always unpause, but this loop continues until the new admin calls `setPauser` to move the role to a new key. A pause halts `deposit`, `requestRedeem`, and `updateAum`, so during the loop the griever can block new deposits, new redemptions, and oracle updates, which lets the price go stale.

The same holds for `redeemManager`, `aumUpdater`, and `backstop`. If the outgoing operator controlled any of them, those powers persist after the admin handover until the new admin rotates each one.

The impact is that an outgoing operator can retain operational powers after an admin handover, and can grief the protocol by repeated pausing, until the new admin rotates the affected roles. No funds are lost, and the new admin can always recover. We classify this issue as informational since it needs a privileged role and is fully recoverable.

Recommendation

We recommend treating role rotation as part of the admin handover. After `acceptAdmin`, the new admin should rotate every operator-controlled role with the existing setters, which are `setPauser`, `setRedeemManager`, `setAumUpdater`, and `setBackstop`, as needed. Document a handover checklist so this step is not missed. Do not fold the `pauser` into `transferAdmin`, because the `pauser` is meant to be a separate role that may be held by a different entity than the admin.

Status: Acknowledged

The client acknowledges the issue stating that the handover checklist has been documented in commit 8c11bbd as part of the protocol's trust assumptions. The new admin is expected to rotate `pauser`, `redeemManager`, `aumUpdater`, `backstop`, and `guardian` using the existing setters following `acceptAdmin` if the outgoing operator held those roles. The `pauser` role is intentionally kept separate from `transferAdmin`, consistent with the recommendation, and the pause grieving loop and its recovery path are explicitly described in the documentation.